principle-viz 套件：經濟學基本圖形

Pin Yue Sung[*†] Ling Tak Douglas Chung[*]

2026-10-03 · v0.10.1 · ·

目錄

1 簡介 2
1.1 功能範圍 2
1.2 閱讀指引 2
2 安裝 3
2.1 系統需求 3
2.2 安裝 uv 3
2.3 安裝 principle-viz 3
2.4 安裝命令列工具 4
2.5 開發環境設定 4
2.6 驗證安裝 4
3 快速開始 4
3.1 基本範例 4
3.2 逐步說明 5
3.3 結果 5
3.4 檢查輸入與求解失敗 6
4 線性市場 6
4.1 直線 6
4.2 均衡 7
4.3 比較靜態 7
4.4 例外 9
5 離散市場 9
5.1 逐單位表 9
5.2 均衡 9
5.3 圖形 10
6 由個人加總市場曲線 11
6.1 加總線性曲線 11
6.2 加總離散表 12
6.3 加總圖 12
7 彈性與總收益 14
7.1 價格彈性 14
7.2 總收益 14
8 福利 15
8.1 市場結果 15
8.2 剩餘 16
8.3 圖形 16
8.4 無謂損失報表 17
9 租稅與補貼 17
9.1 情境 17
9.2 求解 18
9.3 圖形 19
9.4 補貼 20
10 價格管制 21
11 國際貿易 24
12 市場失靈 26
12.1 外部性 26
12.2 共有資源 27
12.3 公共財 28
13 勞動與可貸資金 28
13.1 最低工資 28
13.2 可貸資金 29
14 生產可能曲線 30
14.1 比較利益 31
15 圖形 32
15.1 MarketFigure 32
15.2 標籤與圖層 33
15.3 配色與主題 34
15.4 畫布 36
16 命令列介面 36
16.1 市場參數 36
16.2 指令 36
更新紀錄 38
指令索引 40
參考文獻 42

[*]國立政治大學國際經營與貿易學系，臺北，臺灣。
[†]通訊作者。電子郵件：contact@econ-viz.org

第 1 節　簡介

principle-viz 是經濟學原理的繪圖套件，涵蓋供給與需求、均衡及其移動、租稅與補貼、價格管制、福利、國際貿易、市場失靈、要素市場與生產可能曲線。每個主題都有計算與圖形兩個部分：計算回傳數值，圖形依教科書慣例繪製。

輸出格式包括 PNG、SVG 與 PDF；命令列工具則以 JSON 輸出計算結果。

principle-viz 屬於 EconViz 系列套件，繪圖使用與領域無關的場景與繪製函式庫 mosaickit。同系列的 utility-viz 是個體經濟學繪圖套件，涵蓋效用模型、最適消費組合求解，以及無異曲線、預算線、消費者均衡、需求曲線與 Edgeworth 箱形圖；本套件不包含這些功能。

1.1　功能範圍

principle-viz 處理線性市場，功能分為市場、政策、應用與工具四個部分。計算與圖形可分開使用；同一個結果可以印出、輸出為 JSON，也可以交給 `MarketFigure` 繪製。

1.2　閱讀指引

本手冊依主題分成市場、政策、應用與工具四個部分（參見表 1）：

表 1
章節主題

主題	子主題	說明	章節
市場	線性市場	直線、均衡與曲線移動	第 4 節
	離散市場	逐單位的需求與供給表	第 5 節
	市場曲線加總	個人曲線的水平加總	第 6 節
	彈性與總收益	點彈性、弧彈性與總收益曲線	第 7 節
政策	福利	消費者剩餘、生產者剩餘與無謂損失	第 8 節
	租稅與補貼	稅負楔差與補貼成本	第 9 節
	價格管制	價格上限、價格下限與短缺	第 10 節
應用	國際貿易	自由貿易、關稅與進口配額	第 11 節
	市場失靈	外部性、共有資源與公共財	第 12 節
	勞動與可貸資金	最低工資與政府借款	第 13 節
	生產可能曲線	機會成本、成長與比較利益	第 14 節
工具	圖形	標籤、圖層與配色	第 15 節
	命令列介面	以 JSON 輸出計算結果	第 16 節

初次使用時，依序閱讀第 3 節、第 4 節與第 15 節。之後每章說明一個主題，先介紹計算，再介紹圖形。查詢特定指令的選項時，可直接前往第 16 節。

繪製本手冊各圖的腳本依主題分類，可一次下載：principle-viz-examples.zip。

第 2 節　安裝

2.1　系統需求

principle-viz 需要 Python 3.10 以上版本[1]。唯一的執行期依賴是 mosaickit，它透過 matplotlib 繪製圖形；計算部分只使用標準函式庫。

2.2　安裝 uv

本手冊的指令以 uv[2] 為準。既有專案仍可使用 pip、pipx 或 Poetry；套件 API 不受管理工具影響。

依作業系統執行下列安裝指令。

macOS、Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/
install.ps1 | iex"
```

2.3　安裝 principle-viz

建立專案並加入套件：

```
uv init my-diagrams
cd my-diagrams
uv add principle-viz
uv run python main.py
```

將第 3 節的基本範例存為專案目錄下的 `main.py`，再執行上述指令。`uv add` 記錄專案依賴，`uv run` 使用該專案的 Python 環境。安裝與執行必須使用同一個環境。

若要固定本手冊使用的版本，可在加入依賴時指定版本號：

```
uv add "principle-viz==0.10.1"
```

若使用既有的 Python 虛擬環境，可透過 pip 安裝：

```
python -m pip install -U principle-viz
```

發行套件名稱是 `principle-viz`，Python 匯入名稱是 `principle_viz`：

```
import principle_viz
from principle_viz import solve_equilibrium, MarketFigure
```

`import` 陳述式不得使用連字號。若發生 `ModuleNotFoundError`，檢查執行程式的直譯器是否與安裝套件時使用的環境相同。

0.10.0 版以前，本套件以 `principle-econ` 名稱發布。該發行套件已停止更新；請改裝 `principle-viz`，並將匯入的 `principle_econ` 改為 `principle_viz`。

---

[1] Python 官方網站提供各作業系統的安裝程式：`https://www.python.org/downloads/`。

[2] uv 是 Astral 開發的 Python 套件與專案管理工具，速度快且可一併管理 Python 版本；安裝方式與完整說明見官方文件：`https://docs.astral.sh/uv/`。

2.4　安裝命令列工具

僅使用命令列介面時，可將 principle-viz 安裝為獨立工具（詳見第 16 節）：

```
uv tool install principle-viz
principle-viz equilibrium --demand-intercept 10 --demand-slope -1 \
                          --supply-intercept 2 --supply-slope 1
```

此方式將命令列工具安裝在獨立環境。需要在 Python 程式中匯入套件時，仍須在該專案執行 `uv add principle-viz`。在專案內以 `uv run principle-viz` 呼叫工具，可讓命令列與 Python 程式使用同一版本。

2.5　開發環境設定

```
git clone https://github.com/EconViz/principle-viz.git
cd principle-viz
uv sync
uv run pytest -q
uv run ruff check src tests examples/scripts
```

測試要求至少 90% 的陳述式覆蓋率。範例腳本會將專案圖庫中的所有圖形寫入 `examples/output/`：

```
uv run python examples/scripts/run_all.py
```

2.6　驗證安裝

以下指令確認 Python 能匯入求解與繪圖介面：

```
uv run python -c "import principle_viz; print('OK')"
```

命令列工具以 `uv run principle-viz --help` 確認，會列出所有指令。在伺服器或其他沒有圖形介面的環境中，請以 `save()` 輸出檔案。

第 3 節　快速開始

3.1　基本範例

本章以一個市場說明完整流程，後續各章也沿用這個市場：

$$\text{需求：}\quad p = 10 - Q,$$

$$\text{供給：}\quad p = 2 + Q.$$

例 1

```python
from principle_viz import MarketFigure, line_from_inverse, solve_equilibrium

demand = line_from_inverse(10.0, -1.0)   # p = 10 - Q
supply = line_from_inverse(2.0, 1.0)     # p = 2 + Q
eq = solve_equilibrium(demand, supply)
print(eq.q_star, eq.p_star)              # 4.0 6.0

fig = MarketFigure(x_max=12, y_max=12, title="Basic Equilibrium")
fig.add_curves(demand, supply, q_max=10)
fig.add_equilibrium(eq)
fig.finalize()
fig.save("basic_equilibrium.png")
```

由 $10 - Q = 2 + Q$ 得 $Q^* = 4$、$p^* = 6$。輸出參見圖 1。

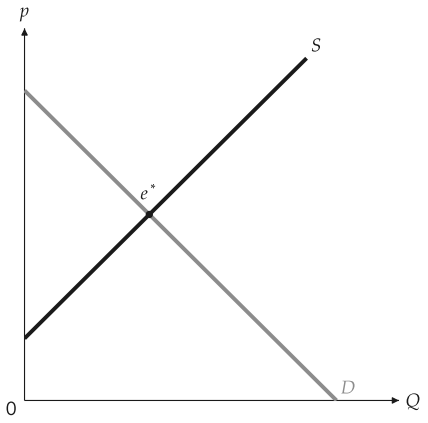


圖 1
本章的市場

3.2　逐步說明

每張圖依序經過描述市場、求解、繪圖、完成並儲存四個步驟。

`line_from_inverse` `line_from_inverse(⟨浮點數⟩, ⟨浮點數⟩)`
描述市場。`line_from_inverse(a, b)` 建立直線 $p = a + bQ$（詳見第 4 節）。

`solve_equilibrium` `solve_equilibrium(⟨需求⟩, ⟨供給⟩)`
求解。`solve_equilibrium()` 等求解函式回傳由數值組成的不可變 dataclass，此時尚未繪圖。

`MarketFigure` `MarketFigure(x_max=⟨浮點數⟩, y_max=⟨浮點數⟩, ...)`
繪圖。`MarketFigure` 是含價格軸與數量軸的正方形圖。座標範圍只控制顯示區域，不參與計算；均衡點未出現在圖中時，檢查 `eq.q_star` 與 `eq.p_star` 是否超出範圍，再調整座標上限。

`MarketFigure.add_*` `add_*` 方法接收曲線與結果，加入對應的圖層；每個方法都回傳圖形本身，可串接呼叫。

`MarketFigure.finalize` 完成並儲存。`finalize()` 隱藏會把陰影面積切成兩半的輔助線；`save()` 依副檔名寫出
`MarketFigure.save` PNG、SVG 或 PDF，並建立不存在的目錄（詳見第 15 節）。

計算與繪圖互不依賴：結果可以不經繪圖直接印出、比較或匯出，同一個結果也可以畫在多張圖上。

3.3　結果

結果是不可變的 dataclass，欄位為浮點數、字串與 tuple。`solve_equilibrium()` 回傳 `EquilibriumResult`：

`q_star` 均衡數量 Q^*。
float

`p_star` 均衡價格 p^*。
float

`is_valid_market` 均衡數量為負時為 `False`；價格為負只會加入一則說明。
bool

notes　說明解的特殊情況。

tuple[*str*, ...]

3.4　檢查輸入與求解失敗

PrincipleVizError　套件拋出的所有例外都繼承自 `principle_viz.exceptions` 中的 `PrincipleVizError`，捕捉它即可一併處理（各例外詳見第 4.4 節）。

兩條直線平行時沒有均衡，`solve_equilibrium()` 拋出 `ParallelLinesError`，不會回傳無效的結果：

```
from principle_viz import line_from_inverse, solve_equilibrium
from principle_viz.exceptions import PrincipleVizError

demand = line_from_inverse(10.0, -1.0)
supply = line_from_inverse(8.0, -1.0)   # parallel to demand

try:
    eq = solve_equilibrium(demand, supply)
except PrincipleVizError as error:
    print(type(error).__name__)
else:
    print(eq.q_star, eq.p_star)

# ParallelLinesError
```

均衡數量為負時不拋出例外，而是將 `is_valid_market` 設為 `False`。使用結果前，先檢查 `is_valid_market` 與 `notes`。

第 4 節　線性市場

4.1　直線

需求或供給曲線以反函數形式表示為直線：

$$p = a + bQ$$

其中 a 為價格截距，b 為斜率（需求 $b < 0$，供給 $b > 0$）。價格一律在縱軸、數量在橫軸，沿用 Marshall (1890) 的畫法。離散的逐單位表（詳見第 5 節）與由個人曲線加總而成的分段線性市場曲線（詳見第 6 節）不是單一直線。

Line　`Line.from_inverse(⟨浮點數⟩, ⟨浮點數⟩)`

`Line.from_standard(⟨浮點數⟩, ⟨浮點數⟩, ⟨浮點數⟩)`

價格—數量平面上的直線，內部儲存為 $Ap + BQ + C = 0$，因此也能表示水平線與垂直線。`from_inverse(a, b)` 建立 $p = a + bQ$；`from_standard(A, B, C)` 建立 $Ap + BQ + C = 0$。套件根目錄另以 `line_from_inverse()` 與 `line_from_standard()` 提供這兩個建構函式。

p_at(q)　數量 q 時的價格。

q_at(p)　價格 p 時的數量。

p_intercept()　$Q = 0$ 時的價格（需求曲線的阻絕價格）。

q_intercept()　$p = 0$ 時的數量。

`slope()` $\mathrm{d}p/\mathrm{d}Q$。

`to_inverse()` $p = a + bQ$ 的 `(a, b)`。

`shifted()` 將截距與斜率分別加上 `delta_intercept`（Δa）與 `delta_slope`（Δb）後的新直線。

```
from principle_viz import line_from_inverse, line_from_standard

demand = line_from_inverse(10.0, -1.0)
print(demand.q_at(4), demand.p_at(3))         # 6.0 7.0
print(demand.p_intercept(), demand.q_intercept())  # 10.0 10.0
print(line_from_standard(1, 1, -10).to_inverse())  # (10.0, -1.0)
```

$(A, B, C) = (1, 1, -10)$ 與 $p = 10 - Q$ 等價。

水平線沒有 $Q(p)$，垂直線沒有 $p(Q)$；呼叫對應的方法會拋出 `NonInvertibleLineError`。

4.2　均衡

`solve_equilibrium` `solve_equilibrium(⟨需求⟩, ⟨供給⟩)`

兩條直線的交點，回傳 `EquilibriumResult`（詳見第 3 節）。兩線平行時拋出 `ParallelLinesError`，重合時拋出 `CoincidentLinesError`。

```
from principle_viz import solve_equilibrium

eq = solve_equilibrium(demand, line_from_inverse(2.0, 1.0))
print(eq)
# EquilibriumResult(q_star=4.0, p_star=6.0,
#                   is_valid_market=True, notes=())
```

由 $10 - Q = 2 + Q$ 得均衡數量 4、均衡價格 6，`is_valid_market` 為 `True`，`notes` 為空。

4.3　比較靜態

`ShiftSpec` `ShiftSpec(delta_intercept=0.0, delta_slope=0.0)`

`ShiftScenario` `ShiftScenario(demand_shift=None, supply_shift=None)`

以反函數形式描述一條曲線的移動：`delta_intercept` 為正時向上平移、為負時向下，`delta_slope` 旋轉曲線。情境可同時移動需求、供給或兩者。兩者都位於 `principle_viz.core.shifts`。

需求增加使需求截距上升；供給增加則使供給截距下降，因為賣方在每個數量下都願意接受較低的價格。

`comparative_statics` `comparative_statics(⟨需求⟩, ⟨供給⟩, ⟨情境⟩)`

求解移動前後的市場，結果包含下列欄位：

`baseline_equilibrium` 移動前的 `EquilibriumResult`。

`shifted_equilibrium` 移動後的 `EquilibriumResult`。

`shifted_market` 移動前後的直線（`baseline_demand`、`shifted_demand` 等）。

`delta_q` 數量的變動。

`delta_p` 價格的變動。

`direction_q` 數量的變動方向：`"left"` 或 `"right"`（未變動時為 `"none"`）。

`direction_p` 價格的變動方向：`"up"` 或 `"down"`（未變動時為 `"none"`）。

```
from principle_viz import comparative_statics
from principle_viz.core.shifts import ShiftScenario, ShiftSpec
```

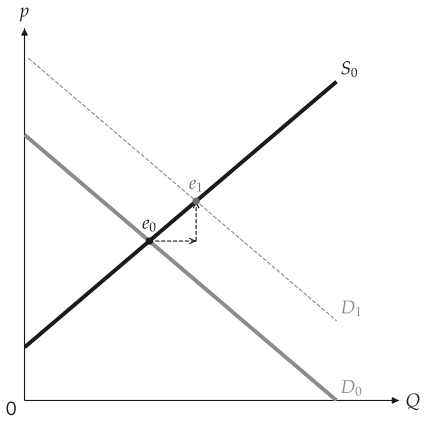


圖 2
需求增加

```
up = ShiftScenario(demand_shift=ShiftSpec(delta_intercept=3.0))
result = comparative_statics(demand, supply, up)
new = result.shifted_equilibrium
print(new.q_star, new.p_star)                    # 5.5 7.5
print(result.direction_q, result.direction_p)  # right up
```

MarketFigure.add_comparative_statics

```
add_comparative_statics(〈結果〉, q_max, *, demand_label="$D_1$",
supply_label="$S_1$")
```

畫出移動的曲線、兩個均衡點，以及由舊均衡指向新均衡的虛線箭頭。在 `add_curves()` 中將原曲線命名為 D_0 與 S_0。需求增加的結果如圖 2，供給減少的結果如圖 3。

```
fig = MarketFigure(x_max=12, y_max=14, title="Increase in Demand")
fig.add_curves(
    demand, supply, q_max=10,
    demand_label="$D_0$", supply_label="$S_0$",
)
fig.add_comparative_statics(result, q_max=10)
fig.finalize()
```

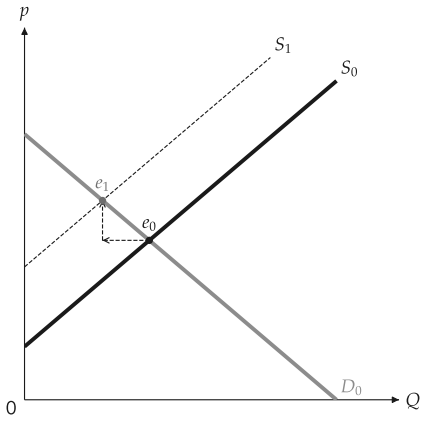


圖 3
供給減少

4.4　例外

套件拋出的所有例外都繼承自 `principle_viz.exceptions` 中的 `PrincipleVizError`，捕捉它即可一併處理。0.10.0 版以前的名稱 `PrincipleEconError` 是同一個類別。各例外的拋出時機如表 2。

表 2　例外類別

例外	拋出時機
`LineError`	無法建立或轉換直線
`NonInvertibleLineError`	對水平線求 $Q(p)$，或對垂直線求 $p(Q)$
`ParallelLinesError`	需求與供給沒有交點
`CoincidentLinesError`	需求與供給是同一條直線
`PolicyError`	租稅、補貼、價格管制或貿易情境無效
`DiscreteMarketError`	離散逐單位表無效（詳見第 5 節）
`AggregationError`、`PiecewiseLinearError`	個人曲線無法加總，或價格超出分段曲線範圍（詳見第 6 節）
`PPFError`	生產可能曲線無效（詳見第 14 節）

第 5 節　離散市場

離散市場逐一列出每個單位：買方對多一單位的願付價格，以及賣方多生產一單位的成本。需求與供給都是階梯函數；均衡是整數個單位，並由一段價格區間支持。

5.1　逐單位表

`DiscreteDemand` `DiscreteDemand(⟨數值⟩)`

`DiscreteSupply` `DiscreteSupply(⟨數值⟩)`

需求表存放邊際願付價格，由第一單位到最後一單位弱遞減；供給表存放邊際成本，弱遞增。順序不符時拋出 `DiscreteMarketError`。

`quantity_at(price)` 價格為 `price` 時的成交單位數；在該價格下無差異的買方或賣方會交易。

`unit_count` 表中的單位數。

`combine(*schedules)` 將多個個人表合併為市場表（詳見第 6 節）。

5.2　均衡

`solve_discrete_equilibrium` `solve_discrete_equilibrium(⟨需求⟩, ⟨供給⟩, *, price_rule="midpoint")`

交易所有價值不低於成本的單位，並找出恰好使該數量成交的價格區間。結果包含下列欄位：

`q_star` 成交單位數。

int

`price_low` 支持均衡的價格區間下限。

float

`price_high` 支持均衡的價格區間上限。

float

`price` 依 `price_rule` 從區間中選出的價格。

float

`price_rule` `MIDPOINT`（預設）、`LOWER` 或 `UPPER`；也可傳入字串 `"midpoint"`、`"lower"`、`"upper"`。

EquilibriumPriceRule

`traded_values` 成交單位的價值。

tuple

`traded_costs` 成交單位的成本。

tuple

`gains_from_trade` 每個成交單位的價值減成本。

tuple

`is_unique_price` 區間是否只有單一價格。

bool

`solve_discrete_market(demand_values, supply_values)` 直接由 tuple 建立兩張表並求解。需求表的值由高到低、供給表的值由低到高，各單位的價值與成本才能依序配對。

```
from principle_viz import (
    DiscreteDemand, DiscreteSupply, solve_discrete_equilibrium,
)

demand = DiscreteDemand((11, 9, 7, 5, 3))
supply = DiscreteSupply((1, 3, 5, 8, 10))
eq = solve_discrete_equilibrium(demand, supply)
print(eq.q_star, eq.price_low, eq.price_high, eq.price)
# 3 5.0 7.0 6.0
print(eq.gains_from_trade)
# (10.0, 6.0, 2.0)
```

成交 3 單位，價格區間為 5 到 7，中點規則回報 6。交易利得合計 $10 + 6 + 2 = 18$。

`compute_discrete_surplus(⟨結果⟩)`

`compute_discrete_surplus`

所選價格下的消費者剩餘與生產者剩餘，包含總額與逐單位數值（`consumer_surplus_by_unit`、`producer_surplus_by_unit`）。上例價格為 6 時，兩者皆為 $5 + 3 + 1 = 9$。

5.3　圖形

`MarketFigure.add_discrete_curves(demand=None, supply=None, *, demand_label="$D$", supply_label="$S$")`

`MarketFigure.add_discrete_curves`

`MarketFigure.add_discrete_equilibrium(⟨結果⟩)`

`MarketFigure.add_discrete_equilibrium`

每個單位畫成區間 $[q, q+1)$ 的一階：起點為實心點（包含），終點為空心點（不包含），並以虛線連到下一階。可只傳入其中一張表。均衡在座標軸上標出 Q^* 與價格區間。

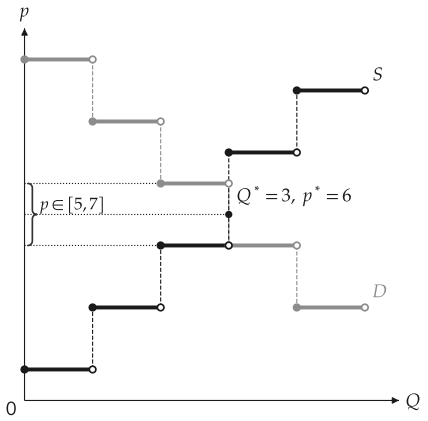


圖 4
離散需求與供給

```
fig = MarketFigure(
    x_max=5.5, y_max=12, title="Discrete Demand and Supply",
)
fig.add_discrete_curves(demand, supply)
fig.add_discrete_equilibrium(eq)
fig.finalize()
```

上例的圖形參見圖 4。

第 6 節　由個人加總市場曲線

市場曲線是個人曲線的水平加總：在每個價格下，市場數量等於所有買方（或賣方）在該價格下選擇的數量之和。買方在阻絕價格以上不購買，賣方在最低價格以下不銷售。線性個人曲線加總後的市場曲線是分段線性的，每多一人進入市場出現一個折點。

6.1　加總線性曲線

`market_demand`　`market_demand(⟨個人曲線⟩)`

`market_supply`　`market_supply(⟨個人曲線⟩, *, p_max)`
加總負斜率（需求）或正斜率（供給）的個人直線。供給加總由最低的最低價格延伸到 `p_max`。斜率方向不符時拋出 `AggregationError`。

`PiecewiseLinear`　`PiecewiseLinear(⟨points⟩)`
通過 `points` (Q, p) 的需求或供給曲線，點依數量排序。若第一點的 $Q = 0$，超出該點的價格下數量為零。

`q_at(p)`　價格對應的數量；超出曲線範圍時拋出 `PiecewiseLinearError`。
`p_at(q)`　數量對應的價格；超出曲線範圍時拋出 `PiecewiseLinearError`。
`points`　所有頂點。
`kinks`　斜率改變的內部頂點。
`domain`　涵蓋的數量範圍。
`price_range`　涵蓋的價格範圍。
`breakpoints()`　頂點價格，由小到大排列。
`integrate_q()`　`p_low` 與 `p_high` 兩個價格之間 $Q(p)$ 下方的面積。

```python
from principle_viz import (
    line_from_inverse, market_demand, market_supply,
)

a = line_from_inverse(10, -2)    # p = 10 - 2Q
b = line_from_inverse(6, -0.5)   # p = 6 - 0.5Q
demand = market_demand((a, b))
print(demand.points)   # ((0.0, 10.0), (2.0, 6.0), (17.0, 0.0))
print(demand.q_at(4))  # 7.0 = Q_A + Q_B = 3 + 4
```

買方 B 在價格 6 進入市場，折點為 $(2, 6)$。價格為 4 時，$Q_A = 3$、$Q_B = 4$，市場需求量為 7。

`solve_piecewise_equilibrium(⟨需求⟩, ⟨供給⟩)`

`piecewise_surplus(⟨需求⟩, ⟨供給⟩, ⟨結果⟩)`

逐段精確求解的市場均衡，以及沿價格積分得到的消費者剩餘與生產者剩餘。

```python
from principle_viz import (
    piecewise_surplus, solve_piecewise_equilibrium,
)

c = line_from_inverse(2, 1)
d = line_from_inverse(5, 0.5)
supply = market_supply((c, d), p_max=10)
eq = solve_piecewise_equilibrium(demand, supply)
print(round(eq.q_star, 3), round(eq.p_star, 3))  # 3.818 5.273
cs, ps = piecewise_surplus(demand, supply, eq)
```

6.2　加總離散表

`DiscreteDemand.combine(*schedules)` 將所有買方的保留價格合併為一張市場表，由高到低排列；`DiscreteSupply.combine()` 合併單位成本，由低到高排列。相同的數值保留為不同單位，合併結果可直接交給 `solve_discrete_equilibrium()`（詳見第 5 節）。輸出參見圖 6。

```python
from principle_viz import DiscreteDemand

first = DiscreteDemand((10, 7, 4))
second = DiscreteDemand((8, 5, 2))
market = DiscreteDemand.combine(first, second)
print(market.values)
# (10.0, 8.0, 7.0, 5.0, 4.0, 2.0)
```

6.3　加總圖

`demand_aggregation_figure(⟨個人曲線⟩, *, price, price_label="$p_1$", link_price=False, ...)`

`supply_aggregation_figure(⟨個人曲線⟩, *, price, p_max, ...)`

`discrete_demand_aggregation_figure(⟨個人曲線⟩, *, price, ...)`

`discrete_supply_aggregation_figure(⟨個人曲線⟩, *, price, ...)`

`discrete_supply_aggregation_figure`

新增：*v0.10.0*

每位個人一個面板，最後是市場面板，左右並排並共用價格軸。`individuals` 將名稱對應到曲線或逐單位表；曲線命名為 D_A、D_B……與 D（或 S_A……與 S）。在 `price` 處以虛線標出 Q_A、Q_B 與 $Q_A + Q_B = Q$。`supply_aggregation_figure()` 另需 `p_max`。

`link_price`

bool · 預設 `False`

讓價格線橫跨所有面板及面板間的空隙，只在第一個面板標出價格。

`theme`

與 `MarketFigure` 相同（詳見第 15.3 節）。

`palette`

與 `MarketFigure` 相同（詳見第 15.3 節）。

`labels`

依圖層 id 覆寫標籤（詳見第 15.2 節）。

`visibility`

依圖層 id 覆寫圖層的顯示狀態（詳見第 15.2 節）。

回傳的 `AggregationFigure` 與 `MarketFigure` 一樣提供 `save()`、`hide()`、`show()`、`configure_label()`、`layer_ids` 與 `label_ids`。

```python
from principle_viz import demand_aggregation_figure

fig = demand_aggregation_figure(
    {"A": a, "B": b}, price=4, link_price=True,
)
fig.save("market_demand.png")
```

`demand_aggregation_figure()` 的輸出參見圖 5。

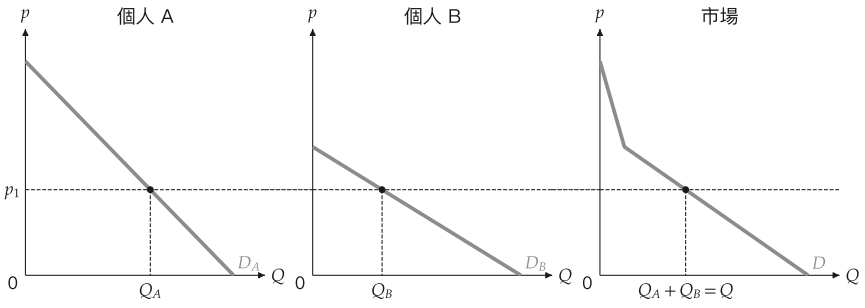


圖 5
水平加總的市場需求

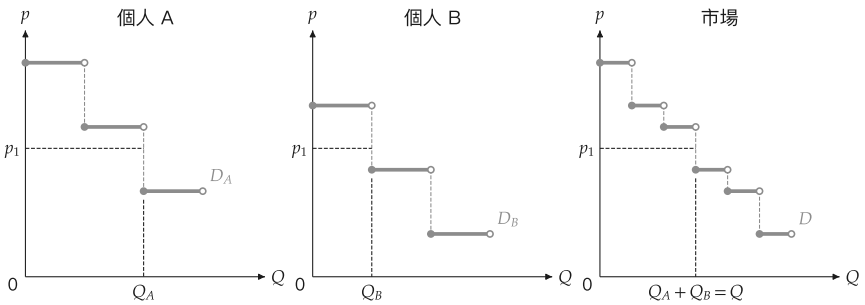


圖 6
合併兩張離散需求表

第 7 節　彈性與總收益

7.1　價格彈性

compute_point_elasticity(⟨需求⟩, ⟨*quantity*⟩)

compute_arc_elasticity(q0, p0, q1, p1)

直線在某數量下的點價格彈性

$$\varepsilon = \frac{\mathrm{d}Q}{\mathrm{d}p} \cdot \frac{p}{Q},$$

以及兩點之間的弧彈性（中點法）

$$\varepsilon = \frac{\Delta Q / \overline{Q}}{\Delta p / \overline{p}},$$

其中 $\overline{Q}$ 與 $\overline{p}$ 為兩點的平均值。負斜率需求的彈性皆為負值。

```
from principle_viz import (
    compute_arc_elasticity, compute_point_elasticity,
line_from_inverse,
)

demand = line_from_inverse(10.0, -1.0)
print(compute_point_elasticity(demand, 4))   # -1.5
print(compute_point_elasticity(demand, 5))   # -1.0 (unit elastic)
print(compute_arc_elasticity(4, 6, 6, 4))    # -1.0
```

$Q = 4$ 處彈性為 -1.5，絕對值大於 1，需求有彈性；$Q = 5$ 為單位彈性，也是 $p = 10 - Q$ 的中點。弧彈性以兩點的平均值計算，因此由 $(4, 6)$ 到 $(6, 4)$ 與反方向的結果相同。

`principle_viz.core.elasticity` 中的 `classify_elasticity(value)` 依絕對值分類：大於 1 為 `"elastic"`，等於 1 為 `"unit_elastic"`，小於 1 為 `"inelastic"`。

7.2　總收益

elasticity_revenue_schedule(⟨需求⟩, *, samples=101)

在線性需求上由 $Q = 0$ 取樣到阻絕數量，回傳每一點的價格、總收益 pQ、彈性與其分類。總收益在單位彈性處（直線的中點）最大。

points　`RevenuePoint` 序列，含 `quantity`、`price`、`total_revenue`、`elasticity` 與 `classification`。

unit_elastic_quantity　直線中點的數量。

unit_elastic_price　直線中點的價格。

maximum_revenue　該點的總收益。

choke_quantity　數量軸截距。

choke_price　價格軸截距。

$p = 12 - Q$ 的總收益最大值為 36，位於 $Q = 6$、$p = 6$。

elasticity_revenue_canvases

`elasticity_revenue_canvases(⟨需求⟩, ⟨結果⟩, *, theme=None, labels=None, visibility=None)`

回傳兩張 mosaickit 畫布，位於 `principle_viz.visuals.revenue`：一張是需求曲線，沿線標出 “Elastic”、“Unit elastic” 與 “Inelastic”，並標記單位彈性點；另一張是總收益對數量的曲線，標記最大值。兩張畫布各有標題。以 `mosaickit.CanvasGrid` 左右並排（參見圖 7）。

```
from mosaickit import CanvasGrid
from principle_viz import PlotTheme, elasticity_revenue_schedule
from principle_viz.visuals.revenue import (
    elasticity_revenue_canvases,
)

demand = line_from_inverse(12.0, -1.0)
schedule = elasticity_revenue_schedule(demand)
canvases = elasticity_revenue_canvases(
    demand, schedule, theme=PlotTheme(),
)
CanvasGrid(canvases, rows=1).save("elasticity_total_revenue.png")
```

第 8 節　福利

福利分析比較某個市場結果與其他可能結果：消費者剩餘、生產者剩餘、政府收入，以及無謂損失。

8.1　市場結果

MarketOutcome

`MarketOutcome(quantity, consumer_price, producer_price, label="")`

一個數量，加上買方支付與賣方收到的價格。課稅或補貼時兩個價格不同，價差乘以數量就是政府收入（補貼為負）。此類別位於 `principle_viz.welfare.surplus`，同模組也提供由各種結果建立市場結果的函式：

outcome_from_equilibrium(eq)

自由市場：兩個價格都是 p^*。

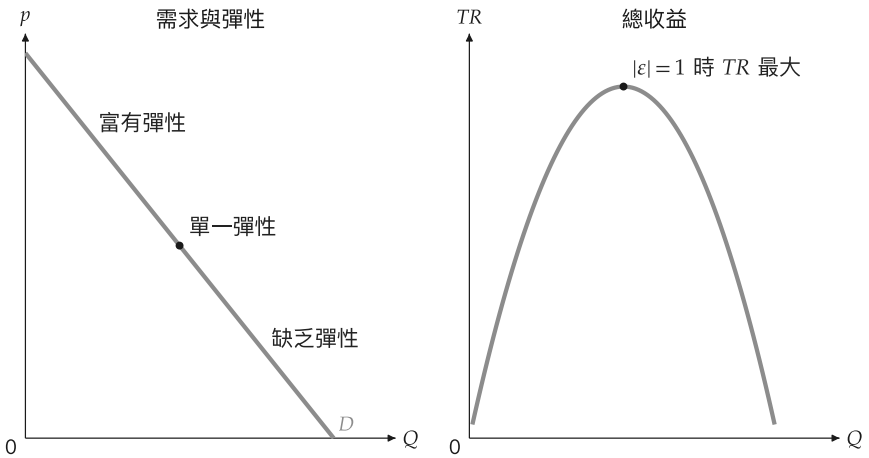


圖 7
彈性與總收益

`outcome_from_tax(tax_eq)` 課稅後（詳見第 9 節）。

`outcome_from_subsidy(subsidy_eq)` 補貼後。

`outcome_from_control(control)` 價格上限或下限之下（詳見第 10 節）。

8.2 剩餘

`compute_surplus` `compute_surplus(⟨需求⟩, ⟨供給⟩, ⟨市場結果⟩, baseline_outcome=None)`
`outcome` 下的福利分解。傳入 `baseline_outcome` 時，以其為基準衡量無謂損失。回傳的 `SurplusResult` 包含下列欄位：

`consumer_surplus` 需求曲線與消費者價格之間的面積。

`producer_surplus` *float* 生產者價格與供給曲線之間的面積。

`tax_revenue` *float* 政府收入（補貼為負）。

`total_surplus` *float* 三者之和。

`deadweight_loss` *float* 相對於基準損失的剩餘。

`polygons` *SurplusPolygons* 各面積的頂點，供繪圖使用。

```
from principle_viz import compute_surplus, solve_equilibrium
from principle_viz.welfare.surplus import outcome_from_equilibrium

eq = solve_equilibrium(demand, supply)
outcome = outcome_from_equilibrium(eq)
surplus = compute_surplus(demand, supply, outcome)
print(surplus.consumer_surplus, surplus.producer_surplus)  # 8.0 8.0
```

$(10-6)\times 4/2 = (6-2)\times 4/2 = 8$；自由市場的無謂損失為 0。

`compare_surplus` `compare_surplus(⟨需求⟩, ⟨供給⟩, baseline_outcome, policy_outcome)`
兩個福利分解及其差異：`baseline`、`policy`、`delta_consumer_surplus`、`delta_producer_surplus`、`delta_tax_revenue`、`delta_total_surplus` 與 `deadweight_loss`。第 9 節以租稅為例使用此函式。

8.3 圖形

`MarketFigure.add_welfare` `add_welfare(⟨結果⟩, *, labels=True, regions=None)`

為消費者剩餘、生產者剩餘、稅收與無謂損失填色並命名：名稱放得下時寫在區塊內，只放得下縮寫時寫 CS、PS、Tax、DWL，否則以引線標示，且不遮住任何線、點或其他文字。

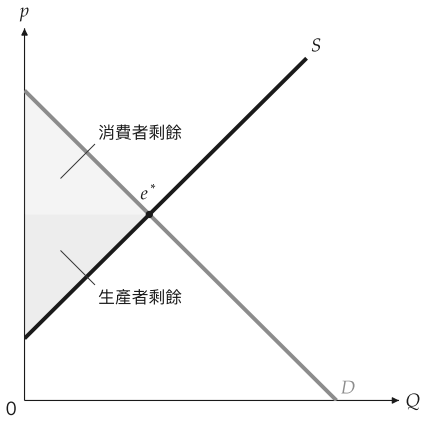


圖 8
均衡下的剩餘

`labels`
bool · 預設 `True`
設為 `False` 時只填色、不命名。

`regions`
iterable | *None* · 預設 `None`
只畫 `"cs"`、`"ps"`、`"tax_revenue"` 與 `"dwl"` 中的部分區塊。

```
fig = MarketFigure(x_max=12, y_max=12)
fig.add_curves(demand, supply, q_max=10)
fig.add_welfare(surplus)
fig.add_equilibrium(eq)
fig.finalize()
```

輸出參見圖 8。

`MarketFigure.add_welfare_transition`
`add_welfare_transition(*, baseline_outcome, policy_outcome, surplus)`
同樣的區塊，再加上基準與政策下數量及價格的輔助線，用於比較前後差異。

8.4　無謂損失報表

`build_dwl_report`
`build_dwl_report(⟨rows⟩)`
將（名稱，基準，政策）三元組（皆為 `SurplusResult`）轉為報表列（`DWLScenarioRow`），包含數量變動、各項剩餘變動與無謂損失。`principle_viz.welfare.report` 中的 `save_dwl_report_csv()` 與 `save_dwl_report_json()` 可將報表寫成檔案。

第 9 節　租稅與補貼

租稅在買方支付的價格與賣方收到的價格之間形成楔差。無論向買方或賣方課徵，楔差、數量與兩個價格都相同 (Mankiw, 2021)。

9.1　情境

`TaxScenario`
`TaxScenario(tax_type, amount, tax_on=TaxOn.PRODUCER, anchor_mode=AnchorMode.NONE)`
一項租稅，位於 `principle_viz.policy.tax`。

`tax_type`　*TaxType*　`FIXED_TAX` 或 `PER_UNIT_TAX`：每單位課徵固定金額，曲線平移 `amount`；`AD_VALOREM_TAX`：依價格課徵比率，曲線旋轉。兩種從量形式的均衡相同。

`amount`　*float*　每單位稅額，或從價稅的稅率（`0.35` 即 35%）；稅率必須大於 -1。

`tax_on`　*TaxOn* · 預設 `PRODUCER`　法定納稅人：`PRODUCER` 使供給上移，`CONSUMER` 使需求下移。

`anchor_mode`　*AnchorMode* · 預設 `NONE`　繪製從價稅時，`NONE` 以截距為軸旋轉曲線，`BASELINE_EQUILIBRIUM` 以未課稅的均衡點為軸。

9.2　求解

TaxType　`TaxType.FIXED_TAX, PER_UNIT_TAX, AD_VALOREM_TAX`
稅的定義，來自 `principle_viz.policy.tax`。

`FIXED_TAX`　*TaxType*　固定金額，使曲線平移 `amount`。

`PER_UNIT_TAX`　*TaxType*　每單位的稅額；均衡與 `FIXED_TAX` 相同。

`AD_VALOREM_TAX`　*TaxType*　按價格計的稅率，使曲線旋轉。

TaxOn　`TaxOn.CONSUMER, PRODUCER`
稅的法律歸宿，來自 `principle_viz.policy.tax`。

`CONSUMER`　*TaxOn*　向買方課徵：需求向下平移。

`PRODUCER`　*TaxOn*　向賣方課徵：供給向上平移。

`solve_tax_equilibrium`　`solve_tax_equilibrium(⟨需求⟩, ⟨供給⟩, ⟨情境⟩)`
課稅後的市場，回傳 `TaxEquilibriumResult`：`q_star`、`consumer_price`、`producer_price`、`tax_wedge`（兩價之差）與 `tax_revenue`（楔差乘以數量）。

```
from principle_viz import solve_tax_equilibrium
from principle_viz.policy.tax import TaxOn, TaxScenario, TaxType

tax = TaxScenario(TaxType.PER_UNIT_TAX, 3.0, TaxOn.PRODUCER)
print(solve_tax_equilibrium(demand, supply, tax))
# TaxEquilibriumResult(q_star=2.5, consumer_price=7.5,
#                      producer_price=4.5, tax_wedge=3.0,
#                      tax_revenue=7.5)
```

`TaxOn.CONSUMER` 的結果相同。稅率為 r 的從價稅使消費者價格等於生產者價格的 $(1+r)$ 倍。

`compare_tax_scenario`　`compare_tax_scenario(⟨需求⟩, ⟨供給⟩, ⟨情境⟩)`
並列課稅前後的市場：`baseline_equilibrium`、`post_tax`、變動量 `delta_q`、`delta_p_consumer`、`delta_p_producer`，以及各自的方向（`"left"`／`"right"`、`"up"`／`"down"`）。

`build_tax_visual_guide`　`build_tax_visual_guide(⟨需求⟩, ⟨供給⟩, ⟨情境⟩)`

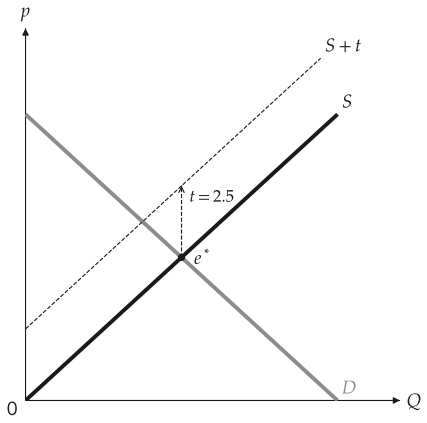


圖 9
向賣方課從量稅

課稅曲線的幾何資訊：`base_curve`、`taxed_curve`、`curve_role`（`"supply"` 或 `"demand"`）與 `transform_kind`（`"shift"` 或 `"rotation"`）。`add_tax_transform()` 依此繪圖。

9.3　圖形

`MarketFigure.add_tax_transform`

`add_tax_transform(⟨需求⟩, ⟨供給⟩, ⟨情境⟩, q_max)`

畫出課稅後的曲線並命名為 $S + t$ 或 $D - t$，以虛線箭頭表示平移或旋轉，並標出未課稅的均衡（參見圖 9、圖 10）。

`MarketFigure.add_tax_comparison`

`add_tax_comparison(⟨結果⟩, *, brace_side="outside", notes=False)`

標出 `compare_tax_scenario()` 結果的楔差：在價格軸上標 p_d（買方）、p_0（課稅前）與 p_s（賣方），並以 “Tax” 括號涵蓋 p_s 到 p_d。

`brace_side`
str · 預設 `"outside"`

括號在座標軸外側（`"outside"`）或繪圖區內（`"inside"`）。

`notes`
bool · 預設 `False`

在座標軸旁說明每個標記。

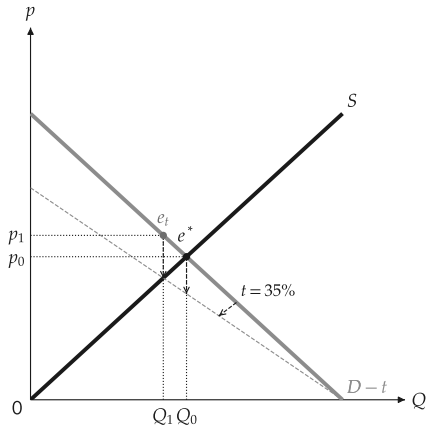


圖 10
向買方課從價稅

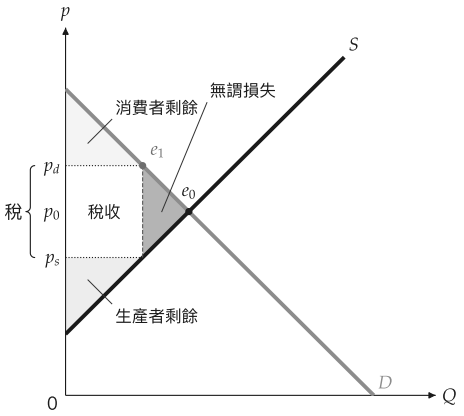


圖 11
課稅下的剩餘與損失

搭配第 8 節的福利區塊（參見圖 11）：

```
from principle_viz import compare_tax_scenario
from principle_viz.welfare.surplus import (
    compare_surplus, outcome_from_equilibrium, outcome_from_tax,
)

eq = solve_equilibrium(demand, supply)
baseline = outcome_from_equilibrium(eq)
taxed = outcome_from_tax(solve_tax_equilibrium(demand, supply, tax))
delta = compare_surplus(demand, supply, baseline, taxed)
print(delta.policy.tax_revenue, delta.deadweight_loss)  # 7.5 2.25

fig = MarketFigure(x_max=12, y_max=12, title="Welfare Under a Tax")
fig.add_curves(demand, supply, q_max=10)
fig.add_welfare(delta.policy)
fig.add_tax_comparison(compare_tax_scenario(demand, supply, tax))
fig.finalize()
```

9.4　補貼

SubsidyScenario　`SubsidyScenario(amount, subsidy_to=SubsidyTo.PRODUCER)`
每單位 `amount` 的補貼（不得為負），發給賣方（`SubsidyTo.PRODUCER`）或買方（`SubsidyTo.CONSUMER`），位於 `principle_viz.policy.subsidy`。

SubsidyTo　`SubsidyTo.PRODUCER, CONSUMER`
補貼的受領者，來自 `principle_viz.policy.subsidy`。

PRODUCER　補貼給賣方。
SubsidyTo

CONSUMER　補貼給買方。
SubsidyTo

solve_subsidy_equilibrium　`solve_subsidy_equilibrium(⟨需求⟩, ⟨供給⟩, ⟨情境⟩)`

compare_subsidy_scenario　`compare_subsidy_scenario(⟨需求⟩, ⟨供給⟩, ⟨情境⟩)`

補貼後的市場（`q_star`、`consumer_price`、`producer_price`、`subsidy_wedge`、`government_expenditure`），以及與自由市場的比較（`baseline_equilibrium`、`post_subsidy`、`delta_q`、`delta_p_consumer`、`delta_p_producer`）。在第 3 節的市場中補貼 2：數量 5，買方支付 5，賣方收到 7，政府支出 10。

`MarketFigure.add_subsidy_comparison`

```
add_subsidy_comparison(⟨結果⟩, *, brace_side="outside", notes=False)
```

以標示稅收楔差的方式標出補貼楔差，加上 “Subsidy” 括號，並標示補貼成本。圖 12 以 `regions=("dwl",)` 只為無謂損失填色，並以 `visibility` 隱藏部分標籤（詳見第 15.2 節）。

```
from principle_viz import (
    SubsidyScenario, SubsidyTo, compare_subsidy_scenario,
)

demand = line_from_inverse(12.0, -1.0)
subsidy = SubsidyScenario(3.0, SubsidyTo.PRODUCER)
comparison = compare_subsidy_scenario(demand, supply, subsidy)
fig = MarketFigure(
    x_max=12, y_max=13, title="Per-Unit Subsidy",
    visibility={
        "market.subsidy.expenditure.label": False,
        "market.subsidy.wedge.label": False,
        "market.subsidy.wedge.mark.p_0": False,
        "market.subsidy.wedge.brace": False,
    },
)
```

第 10 節　價格管制

低於均衡價格的價格上限，或高於均衡價格的價格下限，會產生約束：市場無法結清，成交量等於管制價格下需求量與供給量中較小者。

`PriceControlScenario`

```
PriceControlScenario(control_type, control_price)
```

`PriceControlType`

```
PriceControlType.CEILING | PriceControlType.FLOOR
```

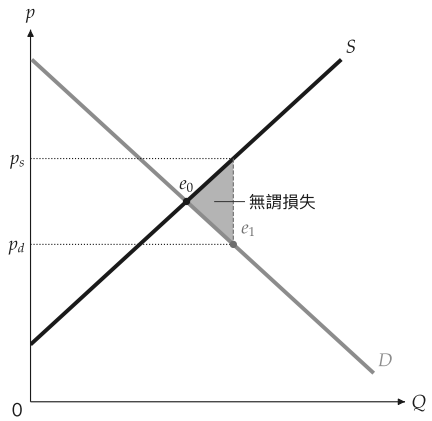


圖 12
從量補貼與無謂損失

位於 control_price 的上限（CEILING）或下限（FLOOR），位於 principle_viz.core.controls。

CEILING 價格上限。

PriceControlType

FLOOR 價格下限。

PriceControlType

evaluate_price_control evaluate_price_control(⟨需求⟩, ⟨供給⟩, ⟨情境⟩)
管制下的市場，回傳 PriceControlResult：

is_binding 管制是否改變市場結果。

bool

traded_quantity 實際成交量。

float

consumer_price 買方支付的價格：有約束時為管制價格，否則為均衡價格。

float

producer_price 賣方收到的價格：有約束時為管制價格，否則為均衡價格。

float

shortage 上限下的超額需求（其餘情況為零）。

float

surplus 下限下的超額供給（其餘情況為零）。

float

baseline_equilibrium 未管制的市場。

EquilibriumResult

```python
from principle_viz import evaluate_price_control
from principle_viz.core.controls import (
    PriceControlScenario, PriceControlType,
)

scenario = PriceControlScenario(PriceControlType.CEILING, 4.0)
ceiling = evaluate_price_control(demand, supply, scenario)
print(ceiling.is_binding, ceiling.traded_quantity, ceiling.shortage)
# True 2.0 4.0
```

上限高於均衡價格（例如 7）時，is_binding 為 False。

MarketFigure.add_price_control add_price_control(⟨結果⟩, *, gap_brace="line")

畫出管制線並命名為 “Price ceiling” 或 “Price floor”，在價格軸上標出 p_c。有約束時另標出 Q_d 與 Q_s，並以括號標示差額：上限下方為 “Shortage”，下限上方為 “Surplus”。gap_brace="axis" 改在數量軸下方標示（參見圖 13、圖 14）。

outcome_from_control() 將結果轉為市場結果，可交給第 8 節的福利函式（參見圖 15）：

```python
from principle_viz.welfare.surplus import (
    compare_surplus, outcome_from_control, outcome_from_equilibrium,
)

eq = solve_equilibrium(demand, supply)
baseline = outcome_from_equilibrium(eq)
controlled = outcome_from_control(ceiling)
```

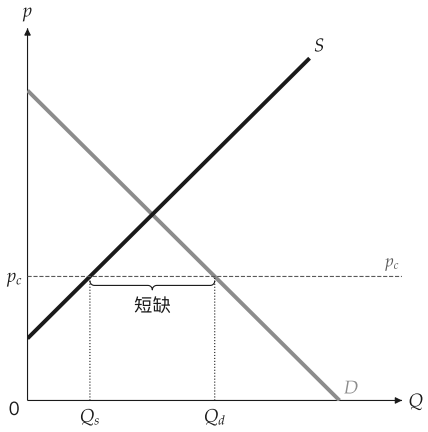


圖 13
有約束的價格上限

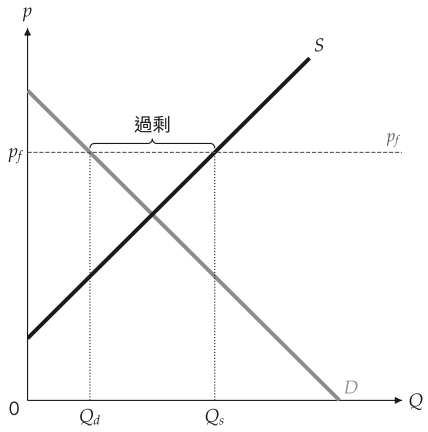


圖 14
有約束的價格下限

```
delta = compare_surplus(demand, supply, baseline, controlled)
fig.add_price_control(ceiling)
fig.add_welfare(delta.policy)
```

價格上限下，無謂損失為未成交的交易利益；部分剩餘由賣方轉移給買方。

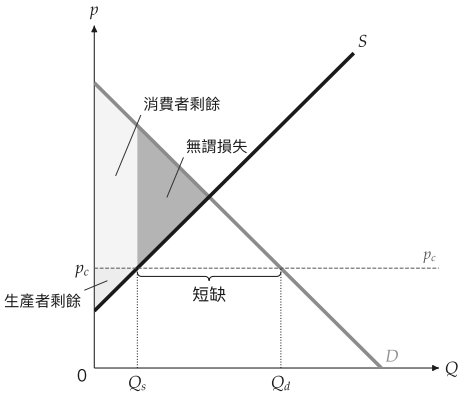


圖 15
上限 3.5 時的福利

第 11 節　國際貿易

小型開放經濟體接受既定的世界價格 p_w。世界價格低於自給自足價格時，進口國內需求與供給的差額；高於自給自足價格時則出口。關稅或進口配額會提高國內價格並減少進口。

`TradeScenario` `TradeScenario(world_price, tariff=0.0, import_quota=None, quota_rent_recipient=QuotaRentRecipient.DOMESTIC)`
世界價格與至多一項政策：每單位 `tariff` 或 `import_quota`，兩者不可同時設定（`PolicyError`）。

`quota_rent_recipient` *QuotaRentRecipient* 配額租的歸屬：國內進口商（`DOMESTIC`）或政府（`GOVERNMENT`），兩者都計入國民剩餘，政府取得的部分列為收入；或外國出口商（`FOREIGN`），配額租流出國外。

`analyze_trade` `analyze_trade(⟨需求⟩, ⟨供給⟩, ⟨情境⟩)`
比較自給自足、自由貿易與政策三種情況。`TradeComparisonResult` 包含三個 `TradeOutcome`（`autarky`、`free_trade`、`policy`），以及政策相對於自由貿易的 `deadweight_loss`。

`domestic_price` 國內市場價格。
`quantity_demanded` 該價格下的國內需求量。
`quantity_supplied` 該價格下的國內供給量。
`imports` 進口量。
`exports` 出口量。
`direction` `TradeDirection`：`IMPORT`、`EXPORT` 或 `AUTARKY`。
`consumer_surplus` 消費者剩餘。
`producer_surplus` 生產者剩餘。
`government_revenue` 政府收入。
`quota_rent` 配額產生的租。
`national_quota_rent` 配額租中留在國內的部分。
`total_surplus` 國民剩餘。
`gains_from_trade` 相對於自給自足的增加。
`is_policy_binding` 關稅或配額是否改變價格。

```
from principle_viz import TradeScenario, analyze_trade

demand = line_from_inverse(12.0, -1.0)   # autarky price 7
scenario = TradeScenario(world_price=4, tariff=2)
result = analyze_trade(demand, supply, scenario)
print(result.free_trade.imports, result.policy.imports)
# 6.0 2.0
print(result.policy.government_revenue, result.deadweight_loss)
# 4.0 4.0
```

關稅 2 時，國內價格為 6，政府收入為 $2 \times 2 = 4$。配額 2 單位時價格同為 6，配額租 4 歸 `quota_rent_recipient` 指定的一方。

`MarketFigure.add_trade` `add_trade(⟨結果⟩)`
畫出世界價格線並命名為 p_w，標出政策價格（$p_w + t$ 或 p_q），在數量軸上標 Q_s 與 Q_d 並於下方加上 “Imports” 或 “Exports” 括號，再以命名的矩形表示關稅收入或配額租（參見圖 16、圖 17、圖 18）。

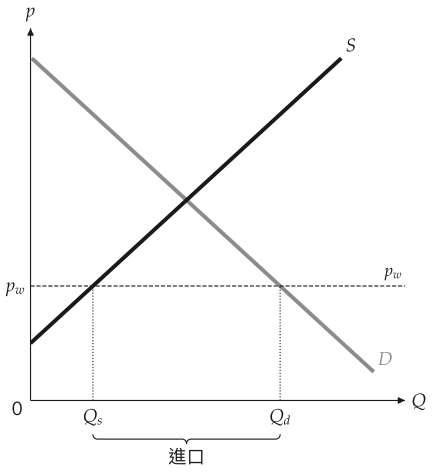


圖 16
自由貿易下的進口

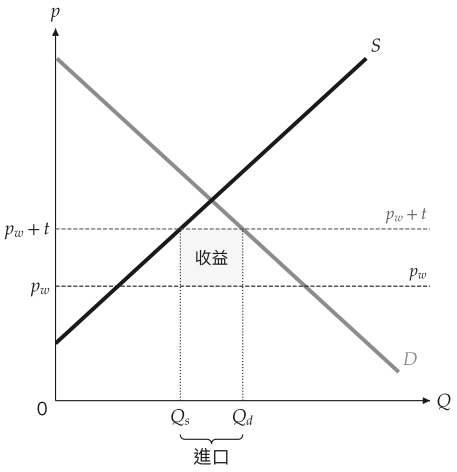


圖 17
進口關稅

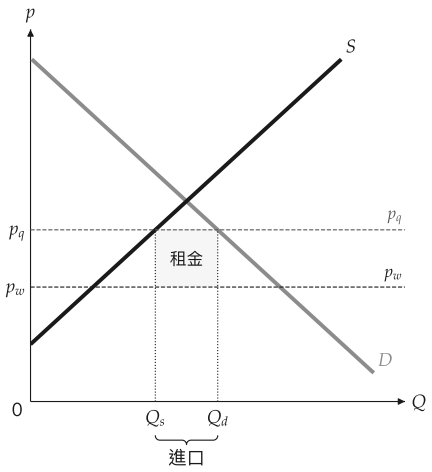


圖 18
有約束的進口配額

第 12 節　市場失靈

12.1　外部性

生產對第三方造成成本，或消費為第三方帶來利益時，市場數量會偏離社會最適數量。在最適數量處課徵等於外部效果的皮古稅或給予皮古補貼，可恢復最適數量 (Pigou, 1920)。

`ExternalityScenario` `ExternalityScenario(marginal_external_cost=0.0, marginal_external_benefit=0.0)`

固定的邊際外部成本（加到供給上得到邊際社會成本），以及／或邊際外部利益（加到需求上得到邊際社會利益）。兩者都不得為負。

`analyze_externality` `analyze_externality(⟨需求⟩, ⟨供給⟩, ⟨情境⟩)`

私人與社會的結果：`private_equilibrium`、`social_equilibrium`、`social_demand`、`social_supply`、`corrective_tax`、`corrective_subsidy`、`quantity_distortion` 與 `deadweight_loss`。

```
from principle_viz import ExternalityScenario, analyze_externality

demand = line_from_inverse(12.0, -1.0)
scenario = ExternalityScenario(marginal_external_cost=2)
result = analyze_externality(demand, supply, scenario)
print(result.private_equilibrium.q_star,
result.social_equilibrium.q_star)
# 5.0 4.0
print(result.corrective_tax, result.deadweight_loss)
# 2.0 1.0
```

`MarketFigure.add_externality` `add_externality(⟨結果⟩)`

畫出社會曲線，在數量軸上標出 Q_m（市場）與 Q^*（最適），在 Q^* 處標出橫跨差距的矯正稅 t 或補貼 s，並為無謂損失填色（參見圖 19、圖 20）。

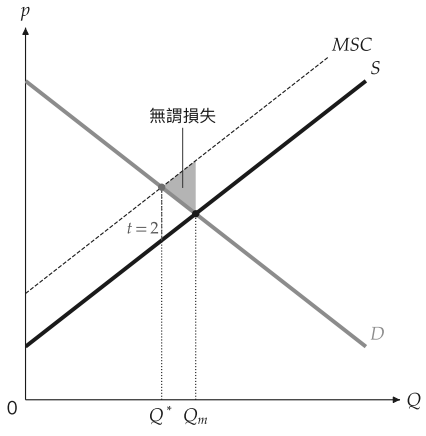


圖 19
負外部性

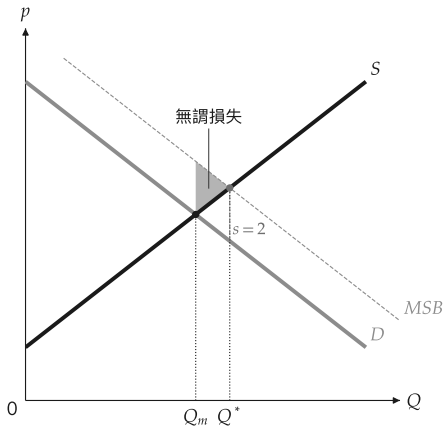


圖 20
正外部性

12.2　共有資源

analyze_common_resource

`analyze_common_resource(⟨benefit⟩, ⟨cost⟩, *, marginal_congestion_cost)`
將擁擠或耗竭視為邊際外部成本：開放取用時，資源會被使用到邊際利益等於私人成本為止，超過有效率的水準 (Hardin, 1968)。結果包含 `open_access_equilibrium`、`efficient_equilibrium`、`social_cost`、`overuse`、`corrective_fee` 與 `deadweight_loss`。

以 $MB = 12 - Q$、$MPC = 2 + Q$、擁擠成本 3 為例，開放取用的使用量為 5 單位，有效率的水準為 3.5，`corrective_fee` 為 3。

MarketFigure.add_common_resource

`add_common_resource(⟨結果⟩)`

畫出社會成本，標出 Q^* 與 Q_{open}，並為無謂損失填色。在 `add_curves()` 中將曲線命名為 *MB* 與 *MPC*。參見圖 21。

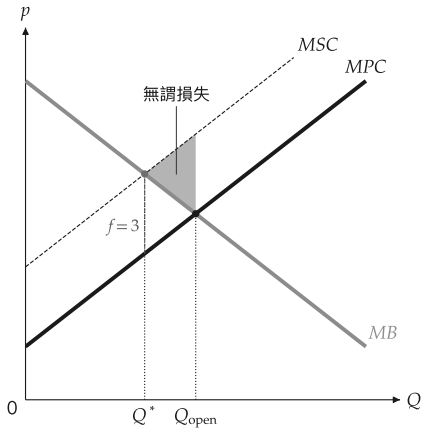


圖 21
共有資源的過度使用

12.3　公共財

`IndividualBenefit` `IndividualBenefit(name, marginal_benefit)`

`analyze_public_good` `analyze_public_good(⟨個人曲線⟩, ⟨cost⟩, *, samples=101)`

每個人都消費公共財的全部數量，因此邊際利益垂直加總。有效率的數量使邊際利益之和等於邊際成本 (Samuelson, 1954)；私人提供則停在最高的個人邊際利益等於邊際成本之處。結果包含 `efficient_quantity`、`efficient_marginal_value`、`private_provision_quantity`、`free_rider_gap` 與取樣點 `points`。

```
from principle_viz import IndividualBenefit, analyze_public_good

result = analyze_public_good(
    (
        IndividualBenefit("$MB_A$", line_from_inverse(8, -1)),
        IndividualBenefit("$MB_B$", line_from_inverse(6, -1)),
    ),
    line_from_inverse(5, 0),  # constant marginal cost of 5
)
print(result.efficient_quantity, result.private_provision_quantity)
# 4.5 3.0
```

`public_good_canvas` `public_good_canvas(⟨結果⟩, *, theme=None, labels=None, visibility=None)`

位於 `principle_viz.visuals.market_failures` 的 mosaickit 畫布，畫出各人的邊際利益、其垂直加總、邊際成本，並在數量軸上標出 Q_p 與 Q^*（參見圖 22）。

第 13 節　勞動與可貸資金

13.1　最低工資

`analyze_minimum_wage` `analyze_minimum_wage(⟨需求⟩, ⟨供給⟩, minimum_wage)`

設有工資下限的競爭勞動市場。`MinimumWageResult` 包含 `equilibrium`、`minimum_wage`、`is_binding`、`labor_demanded`、`labor_supplied`、`employment`（兩者中較小者）、`unemployment`（兩者之差）與 `wage_bill`。

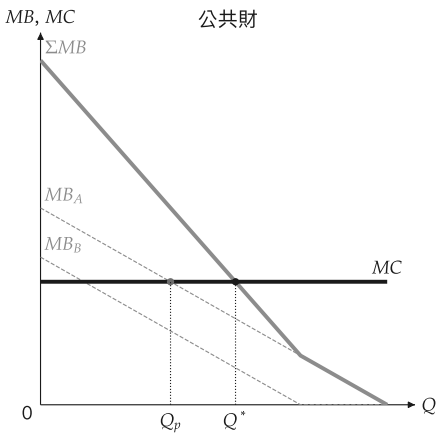


圖 22
邊際利益的垂直加總

```python
from principle_viz import analyze_minimum_wage

labor_demand = line_from_inverse(12, -1)
labor_supply = line_from_inverse(2, 1)
labor = analyze_minimum_wage(
    labor_demand, labor_supply, minimum_wage=9,
)
print(labor.employment, labor.unemployment)  # 3.0 4.0
```

`MarketFigure.add_minimum_wage`

`add_minimum_wage(⟨結果⟩, *, gap_brace="line")`

以價格下限的方式繪製工資下限（詳見第 10 節）：“Minimum wage” 線、工資軸上的 $w_{\min}$，有約束時另標出 L_d 與 L_s 及 “Unemployment” 括號。座標軸命名為 L 與 w，曲線命名為 D_L 與 S_L（參見圖 23）：

```python
fig = MarketFigure(x_max=11, y_max=14, x_label="L", y_label="w")
fig.add_curves(
    labor_demand, labor_supply, q_max=10,
    demand_label="$D_L$", supply_label="$S_L$",
)
fig.add_minimum_wage(labor)
```

`labor_demanded` 為 3，`labor_supplied` 為 7，`employment` 為 3，`unemployment` 為 4。

13.2　可貸資金

`LoanableFundsScenario`

`LoanableFundsScenario(savings_quantity_shift=0.0, investment_quantity_shift=0.0, government_borrowing=0.0)`

儲蓄（供給）與投資（需求）以數量衡量的水平移動。政府借款（不得為負）加入可貸資金的需求。

`analyze_loanable_funds`

`analyze_loanable_funds(⟨savings⟩, ⟨investment⟩, ⟨情境⟩)`

移動前後的可貸資金市場：`baseline_equilibrium`、`shifted_equilibrium`、`shifted_savings`、`shifted_investment_demand`、`private_investment_after`、`interest_rate_change`，以及 `crowding_out`，即利率上升所排擠的民間投資。

```python
from principle_viz import (
    LoanableFundsScenario, analyze_loanable_funds,
```

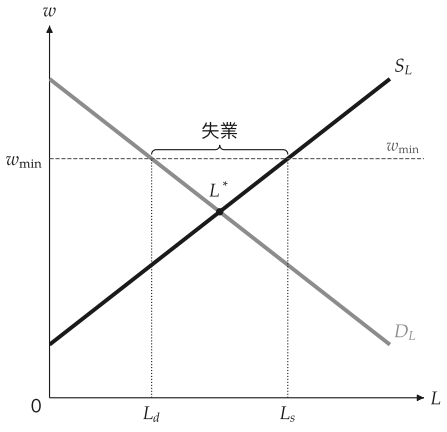


圖 23
有約束的最低工資

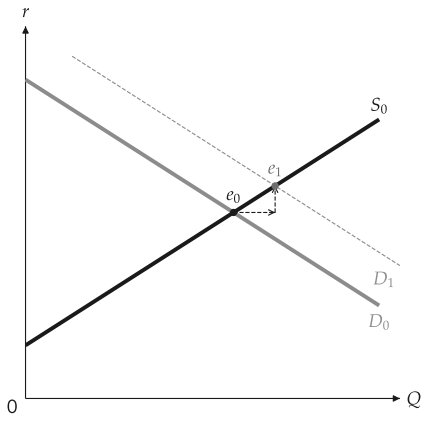


圖 24
政府借款排擠投資

)

```
savings = line_from_inverse(2, 0.5)
investment = line_from_inverse(12, -0.5)
scenario = LoanableFundsScenario(government_borrowing=4)
funds = analyze_loanable_funds(savings, investment, scenario)
print(funds.interest_rate_change, funds.crowding_out)  # 1.0 2.0
```

`interest_rate_change` 為 1.0，`crowding_out` 為 2.0。

`MarketFigure.add_loanable_funds`

`add_loanable_funds(⟨結果⟩)`

畫出移動後的曲線並命名為 D_1 或 S_1，以及兩個均衡點與其間的移動。價格軸命名為 *r*（參見圖 24）。

第 14 節　生產可能曲線

`ProductionPossibilitiesFrontier`

`ProductionPossibilitiesFrontier(x_intercept, y_intercept, curvature=1.0, x_good="Good X", y_good="Good Y")`
通過截距 X 與 Y 的生產可能曲線

$$y = Y(1 - (x/X)^c).$$

曲率 $c = 1$ 為直線（機會成本固定）；$c > 1$ 時曲線向外凸出（機會成本遞增）。截距與曲率必須為正且 $c \geq 1$，否則拋出 `PPFError`。

`y_at(x)` 第一種商品生產 x 單位時 y 的產量。

`opportunity_cost_x(x)` 多生產一單位 x 所放棄的 y，即 $|\mathrm{d}y/\,\mathrm{d}x|$。

`assess(x, y)` 回傳 `PointStatus`：曲線上為 `EFFICIENT`，曲線內為 `INEFFICIENT`，曲線外為 `UNATTAINABLE`。

`PointStatus` `PointStatus.EFFICIENT, INEFFICIENT, UNATTAINABLE`
點相對於邊界的位置。

`EFFICIENT` 在邊界上。

PointStatus

INEFFICIENT 在邊界內側。

PointStatus

UNATTAINABLE 在邊界外側。

PointStatus

```python
from principle_viz import ProductionPossibilitiesFrontier

ppf = ProductionPossibilitiesFrontier(
    10, 8, curvature=2,
    x_good="Consumer goods", y_good="Capital goods",
)
print(ppf.y_at(6), ppf.opportunity_cost_x(6))  # 5.12 0.96
print(ppf.assess(4, 3), ppf.assess(7, 6))
# PointStatus.INEFFICIENT PointStatus.UNATTAINABLE
```

analyze_ppf `analyze_ppf(⟨生産可能曲線⟩, points=(), *, samples=101)`

ppf_canvas `ppf_canvas(⟨結果⟩, *, theme=None, labels=None, visibility=None)`
在曲線上取樣，並評估以 `(x, y, label)` 三元組給定的點；`principle_viz.visuals.ppf` 中的 `ppf_canvas()` 畫出曲線、為可達成的區域填色並標出各點（參見圖 25）。

```python
from principle_viz import analyze_ppf
from principle_viz.visuals.ppf import ppf_canvas

points = ((6, ppf.y_at(6), "A"), (4, 3, "B"), (7, 6, "C"))
result = analyze_ppf(ppf, points=points)
ppf_canvas(result).save("ppf_points.png")
```

PPFGrowthScenario `PPFGrowthScenario(x_growth_rate=0.0, y_growth_rate=0.0)`

analyze_ppf_growth `analyze_ppf_growth(⟨生産可能曲線⟩, ⟨情境⟩, *, samples=101)`

ppf_growth_canvas `ppf_growth_canvas(⟨結果⟩, *, theme=None, labels=None, visibility=None)`
經濟成長使各截距乘以一加上對應的成長率；結果包含 `baseline` 與 `shifted` 兩條曲線及其取樣點。`ppf_growth_canvas()` 畫出兩條曲線並命名為 PPF_0 與 PPF_1（參見圖 26）。

14.1 比較利益

compare_linear_ppfs `compare_linear_ppfs(name_a, frontier_a, name_b, frontier_b)`
比較兩條直線型生產可能曲線：各生產者生產 x 的機會成本，以及各商品的比較利益歸屬（成本相同時為 `"tie"`）。曲線型的生產可能曲線會拋出 `PPFError`。

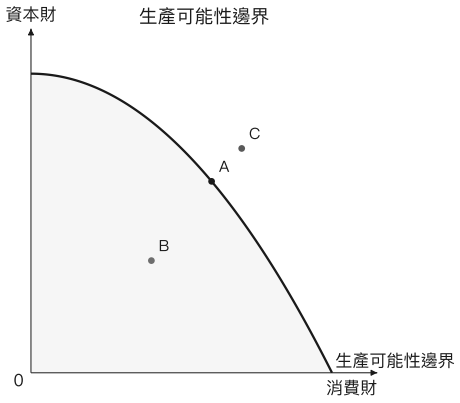


圖 25
有效率與無效率的點

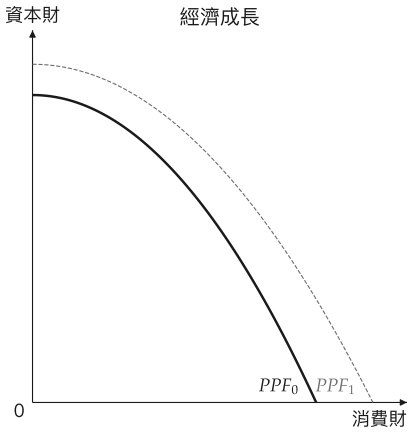


圖 26
消費財與資本財成長

```
from principle_viz import compare_linear_ppfs

ann = ProductionPossibilitiesFrontier(10, 5)  # x costs 0.5 y
bob = ProductionPossibilitiesFrontier(6, 6)   # x costs 1 y
result = compare_linear_ppfs("Ann", ann, "Bob", bob)
print(result.comparative_advantage_x, result.comparative_advantage_y)
# Ann Bob
```

x 的比較利益屬於 Ann（0.5 對 1），y 屬於 Bob（1 對 2）。

第 15 節　圖形

本手冊的每張圖都是 `MarketFigure`，或由 `ppf_canvas()` 等函式建立的 mosaickit 畫布。兩者的繪圖區為正方形，座標軸帶箭頭、無刻度與格線；曲線名稱標在末端，面積名稱寫在區塊內，數值標在座標軸上，文字不遮住線、點、區塊或其他文字。

15.1　MarketFigure

`MarketFigure`

```
from principle_viz import MarketFigure

fig = MarketFigure(
    x_max=20.0,
    y_max=20.0,
    x_label="Q",
    y_label="p",
    title="Market Diagram",
    theme=None,       # PlotTheme
    palette=None,     # "default", "nord", ...
    labels=None,      # {layer id: Label}
    visibility=None,  # {layer id: bool}
)
```

mosaickit 畫布上的市場圖，座標軸由 0 延伸到 `x_max` 與 `y_max`。座標軸名稱除非是文字，否則以 LaTeX 數學排版；`title` 為空字串時不畫標題。

`theme`　顏色與線寬（詳見第 15.3 節）。

`palette`　`PlotTheme.from_palette(palette)` 的簡寫；與 `theme` 同時給定時以 `theme` 為準。

`labels`　依圖層 id 覆寫內建標籤（詳見第 15.2 節）。

`visibility`　依圖層 id 顯示或隱藏圖層（詳見第 15.2 節）。

MarketFigure.add_curves `add_curves(⟨需求⟩, ⟨供給⟩, q_max, demand_label="$D$", supply_label="$S$")`

MarketFigure.add_equilibrium `add_equilibrium(⟨結果⟩, label="$e^*$", color=None)`

由 $Q = 0$ 到 `q_max` 畫出需求與供給並在末端命名；以實心點與標籤標出均衡。

各主題另有專屬方法，說明見對應章節（參見表 3）。

表 3 `MarketFigure` 的主題方法

方法	章節
`add_comparative_statics`	第 4.3 節
`add_discrete_curves`、`add_discrete_equilibrium`	第 5 節
`add_welfare`、`add_welfare_transition`	第 8 節
`add_tax_transform`、`add_tax_comparison`、`add_subsidy_comparison`	第 9 節
`add_price_control`	第 10 節
`add_trade`	第 11 節
`add_externality`、`add_common_resource`	第 12 節
`add_minimum_wage`、`add_loanable_funds`	第 13 節

MarketFigure.finalize `finalize(legend=False)`

MarketFigure.save `save(⟨路徑⟩, dpi=150)`

`finalize()` 隱藏會把福利區塊切成兩半的輔助線（數值仍標在座標軸上），只有在 `legend=True` 時才加入圖例。`save()` 依副檔名寫出 PNG、SVG 或 PDF，並建立不存在的目錄。`close()` 不做任何事，僅為相容而保留。

MarketFigure.add_layer `add_layer(⟨layer⟩)`

MarketFigure.add_layers `add_layers(⟨layers⟩)`

加入任意 mosaickit 圖層，用於主題方法未提供的註記。同一數值的座標軸標記會取代先前的標記。完成的場景可由 `fig.scene` 取得。

MarketFigure.add_metrics `add_metrics(⟨數值⟩, *, title=None, location="upper right")`

以名稱與數值組成的文字框，供筆記本與除錯使用。教學用圖不放此框，數值應寫在正文中。

15.2　標籤與圖層

圖中的每條線、每個點、每個區塊與每段文字都是一個圖層，各有固定的 id，例如 `market.demand`、`market.demand.label` 或 `market.welfare.dwl`。id 依圖形結構命名：座標軸為 `axes.*`，各元素為 `market.<部分>`，命名該元素的文字再加上 `.label` 後綴。`fig.layer_ids` 列出所有圖層；`fig.label_ids` 列出可設定的文字（標籤、座標軸標記與括號）。

Label `Label(text=None, visible=None, offset=None)`

覆寫一個內建標籤。值為 `None` 的欄位保留套件的預設值。指定 `offset`（單位為點）時，標籤不再自動放置，改為精確移動該距離。

新增：*v0.10.1*

`MarketFigure.configure_label` `configure_label(⟨圖層 id⟩, label=None, *, text=None, visible=None, offset=None)`

`MarketFigure.configure_layer` `configure_layer(⟨圖層 id⟩, *, visible)`

改名、隱藏或移動一個標籤（id 可省略結尾的 `.label`），或顯示、隱藏任一圖層。`hide(*ids)` 與 `show(*ids)` 一次切換多個圖層。每個方法都回傳圖形本身。

新增：*v0.10.1*

同樣的覆寫也可在建立圖形時以 `labels=` 與 `visibility=` 對應表傳入，之後加入的圖層也會套用。加總圖與各畫布函式（`ppf_canvas()`、`public_good_canvas()` 等）接受同樣的兩個參數。

參見圖 27：

```
from principle_viz import Label, MarketFigure

fig = MarketFigure(
    x_max=12, y_max=12,
    labels={
        "market.demand.label": Label(text="Demand"),
        "market.supply.label": Label(text="Supply"),
    },
)
fig.add_curves(demand, supply, q_max=10)
fig.add_equilibrium(eq)
fig.configure_label("market.equilibrium", text="$e^*$")
fig.hide("axes.origin.label")
fig.finalize()
```

15.3　配色與主題

`ColorModel` 為每個經濟角色指定顏色；`PlotTheme` 再加上線寬與座標軸選項，並將兩者編譯為 mosaickit 主題。內建的四種配色如表 4。

`ColorModel` `ColorModel(name, axis_color, label_color, demand_color, supply_color, ...)`

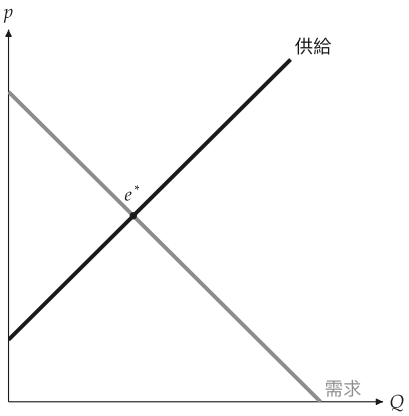


圖 27
改名的曲線與均衡點

具名的顏色角色：`axis_color`、`label_color`、`demand_color`、`supply_color`、`baseline_color`、`shifted_color`、`tax_color`、`control_color`、`cs_color`、`ps_color`、`tax_revenue_color`、`dwl_color` 與 `arrow_color`。每個值可為 `"#hex"` 或 `"blue"` 等 mosaickit 調色盤名稱。

表 4
內建配色

名稱	特色
`default`	mosaickit 色相：需求藍、供給紅、無謂損失青；剩餘沿用曲線色相，透明度 15%
`colorblind`	色盲友善的質性配色
`nord`	Nord 配色
`monochrome`	黑、白與灰階，供印刷使用

`list_color_models()` 回傳所有名稱，`get_color_model(name)` 回傳對應的配色；四種配色也以 `DEFAULT_COLOR_MODEL`、`COLORBLIND_COLOR_MODEL`、`NORD_COLOR_MODEL` 與 `MONOCHROME_COLOR_MODEL` 匯出。

PlotTheme

```
PlotTheme(color_model=..., demand_linewidth=3.5,
supply_linewidth=3.5, ...)
PlotTheme.from_palette(〈名稱〉, **options)
```

配色加上線寬（`demand_linewidth`、`supply_linewidth`、`shifted_linewidth`、`tax_linewidth`、`arrow_linewidth`、`dashed_linewidth`）、`equilibrium_marker_size`，以及開關 `show_grid`、`show_ticks`、`show_axis_arrows` 與 `show_origin_label`。`from_palette()` 以內建配色為起點，並可設定上述任一欄位。

以 `palette` 參數選用配色（參見圖 28）：

```
fig = MarketFigure(x_max=12, y_max=12, palette="monochrome")
```

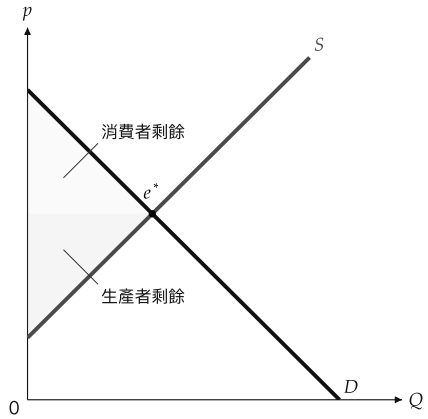


圖 28
`monochrome` 配色

15.4　畫布

Canvas mosaickit 的 `Canvas`，並具備與 `MarketFigure` 相同的 `labels=`、`visibility=`、`configure_label()`、`configure_layer()`、`hide()`、`show()`、`layer_ids` 與 `label_ids`。生產可能曲線、公共財與總收益各章的畫布函式都回傳此類別；以 `save()` 儲存，或以 `mosaickit.CanvasGrid` 組合多張畫布。

第 16 節　命令列介面

`principle-viz` 指令在命令列執行本手冊中的計算，並以 JSON 印出結果；`--output` 會另外寫成檔案。此指令不繪圖。可將其安裝為工具（詳見第 2.4 節），或在專案中於指令前加上 `uv run`。

```
principle-viz equilibrium \
  --demand-intercept 10 --demand-slope -1 \
  --supply-intercept 2 --supply-slope 1
```

```
{
  "q_star": 4.0,
  "p_star": 6.0,
  "is_valid_market": true,
  "notes": []
}
```

16.1　市場參數

多數指令以反函數形式 $p = a + bQ$ 的兩條直線描述市場：

`--demand-intercept` 需求的價格截距 a。

`--demand-slope` 需求的斜率 b。

`--supply-intercept` 供給的價格截距 a。

`--supply-slope` 供給的斜率 b。

`--output` 另將 JSON 寫入此檔案，並建立所需目錄。

16.2　指令

各指令對應的計算與選項如表 5；每個指令的計算說明見對應章節。

表 5 命令列指令

指令	計算與其他選項	章節
`equilibrium`	線性均衡	第 4 節
`shift`	比較靜態：`--demand-delta-intercept`、`--demand-delta-slope`、`--supply-delta-intercept`、`--supply-delta-slope`	第 4.3 節
`discrete`	離散市場：`--demand-values`、`--supply-values`、`--price-rule {midpoint,lower,upper}`	第 5 節
`elasticity`	點彈性：`--intercept`、`--slope`、`--quantity`；加上 `--q1`、`--p1` 計算弧彈性	第 7 節
`revenue`	彈性與總收益表：`--samples`	第 7 節

指令	計算與其他選項	章節
`welfare`	剩餘：`--policy {baseline,tax,subsidy,control}` 與該政策的選項	第 8 節
`report-dwl`	單列無謂損失報表：`--policy {tax,subsidy,control}`、`--csv`	第 8 節
`tax`	`--tax-type {fixed,per_unit,ad_valorem}`、`--amount`、`--tax-on {consumer,producer}`	第 9 節
`subsidy`	`--amount`、`--subsidy-to {consumer,producer}`	第 9 節
`controls`	`--control-type {ceiling,floor}`、`--control-price`	第 10 節
`trade`	`--world-price`、`--tariff`、`--import-quota`、`--quota-rent-recipient`	第 11 節
`externality`	`--external-cost`、`--external-benefit`	第 12 節
`common-resource`	`--congestion-cost`	第 12 節
`public-good`	`--benefit-intercepts`、`--benefit-slopes`、`--cost-intercept`、`--cost-slope`、`--samples`	第 12 節
`minimum-wage`	`--labor-demand-*`、`--labor-supply-*`、`--minimum-wage`	第 13 節
`loanable-funds`	`--savings-*`、`--investment-*`、`--savings-shift`、`--investment-shift`、`--government-borrowing`	第 13 節
`ppf`	`--x-intercept`、`--y-intercept`、`--curvature`、`--x-good`、`--y-good`、`--x-growth`、`--y-growth`、`--samples`	第 14 節

`principle-viz <command> --help` 列出指令的所有選項。

```
principle-viz tax \
  --demand-intercept 10 --demand-slope -1 \
  --supply-intercept 2 --supply-slope 1 \
  --tax-type per_unit --amount 1 --tax-on producer
```

輸出包含未課稅的均衡（`baseline_equilibrium`）、課稅後的均衡（`post_tax`）與各項變動，與 `compare_tax_scenario()` 的回傳值相同：本例數量為 3.5，消費者價格 6.5，生產者價格 5.5。

更新紀錄

本章列出影響套件使用的版本變更，包含以 `principle-econ` 名稱發布的版本，不含僅修改文件的版本；各條目依 l3doc 慣例標註於它所描述的功能旁，頁碼即該功能的實際頁碼。

v0.10.1 (2026-10-03)

一般：CI 測試 Python 3.10–3.13 ····· 2

`demand_aggregation_figure`：市場曲線在折點處標出當時有效的需求 ····· 12

`Label`：每個內建標籤都可改名、隱藏或移動：新增 `Label`、`labels=`、`configure_label()` 與 `label_ids` ····· 33

`MarketFigure.configure_layer`：每個圖層都可隱藏或顯示：新增 `visibility=`、`configure_layer()`、`hide()`、`show()` 與 `layer_ids` ····· 33

v0.10.0 (2026-10-03)

一般：由 `principle-econ` 更名為 principle-viz，加入 EconViz 系列；Python 套件名稱改為 `principle_viz` ····· 2

圖形改以 mosaickit 繪製（`mosaickit>=0.5.1,<0.6.0`） ····· 2

專案工具由 Poetry 改為 uv ····· 2

`principle-viz`：唯一的執行期依賴為 mosaickit ····· 3

`add_comparative_statics`：只重畫移動的曲線；移動後的曲線命名為 D_1 / S_1（`demand_label` / `supply_label`） ····· 7

移動箭頭改為黑色細虛線 ····· 7

`MarketFigure.add_discrete_curves`：可只傳入需求表、只傳入供給表，或兩者都傳入 ····· 9

空心端點改為不透明的白色填色，並以細虛線連到下一階 ····· 9

`market_demand`：個人線性曲線水平加總為市場需求與供給，並新增 `PiecewiseLinear`、`solve_piecewise_equilibrium()` 與 `piecewise_surplus()` ····· 11

`DiscreteDemand.combine`：離散表新增 `combine()` 與 `quantity_at()` ····· 11

`demand_aggregation_figure`：需求與供給（線性與離散）的**個人 | 個人 | 市場**圖，回傳 `AggregationFigure` ····· 12

`link_price=True` 讓價格線橫跨所有面板 ····· 12

`MarketFigure.add_welfare`：以文字命名各區塊（“Consumer surplus”、“CS” 等），寫在區塊內、改用縮寫或以引線標示；`labels=False` 只填色 ····· 15

消費者剩餘與生產者剩餘沿用需求與供給的色相；稅收只標示文字、不填色 ····· 15

`MarketFigure.add_tax_transform`：課稅後的曲線命名為 $S + t$ / $D - t$；旋轉標籤移到箭頭尾端 ····· 17

`MarketFigure.add_tax_comparison`：在價格軸上標出 p_d、p_0 與 p_s，並加上 “Tax” 括號；新增 `brace_side` 與 `notes` ····· 19

`MarketFigure.add_subsidy_comparison`：補貼楔差加上 “Subsidy” 括號，並標示補貼成本 · 20

`MarketFigure.add_price_control`：有效的管制直接命名管制線，標出 p_c、Q_d 與 Q_s，並在管制線上以括號標示 “Shortage” 或 “Surplus”；`gap_brace="axis"` 改在數量軸下方標示 ····· 21

`MarketFigure.add_trade`：標出 Q_s / Q_d，並在數量軸下方以括號標示 “Imports” / “Exports”；政策價格標為 $p_w + t$ 或 p_q ····· 24

`MarketFigure.add_externality`：社會曲線命名為 *MSC* / *MSB*；Q_m 與 Q^* 標在數量軸上；矯正稅與補貼改為橫跨 Q^* 處的差距 ····· 26

`MarketFigure.add_minimum_wage`：以價格下限的方式繪製：“Minimum wage” 線、$w_{\min}$、L_d / L_s 與 “Unemployment” 括號 ····· 28

`MarketFigure`：圖形改為正方形（150 dpi 下 6 × 6 英吋），背景為不透明白色；直線曲線止於價格軸的 92% ····· 32

曲線名稱標在可見端點旁，不再放進圖例 ···· 32

座標軸名稱預設為 p 與 Q，以斜體數學字放在箭頭外側；標題不再使用粗體 ····· 32

均衡與政策標籤由 mosaickit 放在點旁；擁擠處的數量改標在數量軸上 ····· 32

`MarketFigure.finalize`：預設不加圖例；`finalize(legend=True)` 才會加入 ····· 32

`ColorModel`：`default` 配色改用 mosaickit 的色相：需求藍、供給紅、無謂損失青 ····· 34

`PlotTheme`：供需曲線寬 3.5 pt、座標軸 1.0 pt、均衡點直徑 6.5 pt ····· 34

v0.1.0 (2026-04-25)

一般：以 `principle-econ` 名稱首次發布：線性市場、租稅、福利、繪圖與命令列介面 ····· 2

`TaxScenario`：`fixed`、`per_unit` 與 `ad_valorem` 三種租稅，法定納稅人可為買方或賣方 ····· 17

`ColorModel`：`default`、`colorblind`、`nord` 與 `monochrome` 四種配色 ····· 34

`principle-viz`：命令列介面，輸出 JSON ··· 36

指令索引

A

analyze_common_resource 27
analyze_externality 26
analyze_loanable_funds 29
analyze_minimum_wage 28
analyze_ppf 31
analyze_ppf_growth 31
analyze_public_good 28
analyze_trade 24

B

build_dwl_report 17
build_tax_visual_guide 18

C

Canvas 36
ColorModel 34
comparative_statics 7
compare_linear_ppfs 31
compare_subsidy_scenario 20
compare_surplus 16
compare_tax_scenario 18
compute_arc_elasticity 14
compute_discrete_surplus 10
compute_point_elasticity 14
compute_surplus 16

D

demand_aggregation_figure 12
discrete_demand_aggregation_figure 12
discrete_supply_aggregation_figure 13
DiscreteDemand 9
DiscreteSupply 9

E

elasticity_revenue_canvases 15
elasticity_revenue_schedule 14
evaluate_price_control 22
ExternalityScenario 26

I

IndividualBenefit 28

L

Label 33
Line 6
line_from_inverse 5
LoanableFundsScenario 29

M

market_demand 11
market_supply 11
MarketFigure 5, 32
MarketFigure.add_* 5
MarketFigure.add_common_resource 27
MarketFigure.add_comparative_statics 8
MarketFigure.add_curves 33
MarketFigure.add_discrete_curves 10
MarketFigure.add_discrete_equilibrium · 10
MarketFigure.add_equilibrium 33
MarketFigure.add_externality 26
MarketFigure.add_layer 33
MarketFigure.add_layers 33
MarketFigure.add_loanable_funds 30
MarketFigure.add_metrics 33
MarketFigure.add_minimum_wage 29
MarketFigure.add_price_control 22
MarketFigure.add_subsidy_comparison 21
MarketFigure.add_tax_comparison 19
MarketFigure.add_tax_transform 19
MarketFigure.add_trade 24
MarketFigure.add_welfare 16
MarketFigure.add_welfare_transition 17
MarketFigure.configure_label 34
MarketFigure.configure_layer 34
MarketFigure.finalize 5, 33
MarketFigure.save 5, 33

MarketOutcome 15

P

piecewise_surplus 12
PiecewiseLinear 11
PlotTheme 35
PointStatus 30
ppf_canvas 31
ppf_growth_canvas 31
PPFGrowthScenario 31
PriceControlScenario 21
PriceControlType 21
PrincipleVizError 6
ProductionPossibilitiesFrontier 30
public_good_canvas 28

S

ShiftScenario 7
ShiftSpec 7
solve_discrete_equilibrium 9
solve_equilibrium 5, 7
solve_piecewise_equilibrium 12
solve_subsidy_equilibrium 20
solve_tax_equilibrium 18
SubsidyScenario 20
SubsidyTo 20
supply_aggregation_figure 12

T

TaxOn 18
TaxScenario 17
TaxType 18
TradeScenario 24

參考文獻

Hardin, G. (1968). The Tragedy of the Commons. *Science*, *162*(3859), 1243–1248. https://doi.org/10.1126/science.162.3859.1243

Mankiw, N. G. (2021). *Principles of Economics* (9th ed.). Cengage Learning.

Marshall, A. (1890). *Principles of Economics*. Macmillan.

Pigou, A. C. (1920). *The Economics of Welfare*. Macmillan.

Samuelson, P. A. (1954). The Pure Theory of Public Expenditure. *The Review of Economics and Statistics*, *36*(4), 387–389. https://doi.org/10.2307/1925895